

UNIT II

Chapter1 Programming with PHP

Conditionals:

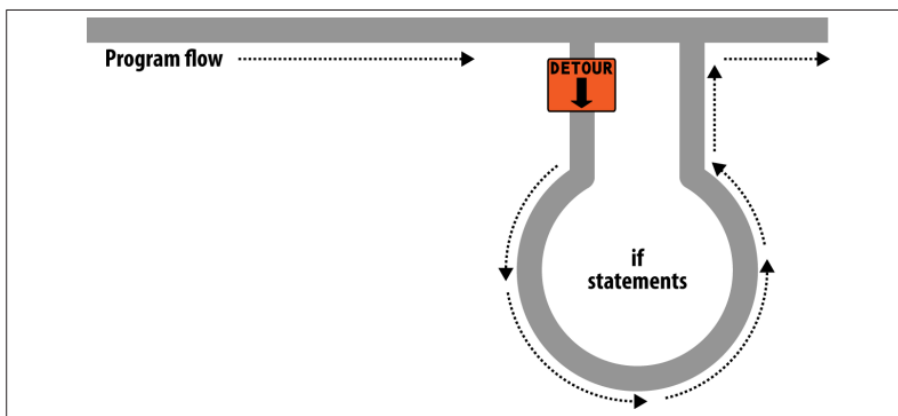
Conditionals alter program flow. They enable you to ask questions about certain things and respond to the answers you get in different ways. Conditionals are central to dynamic web pages—the goal of using PHP in the first place—because they make it easy to create different output each time a page is viewed.

There are three types of non-looping conditionals: the if statement, the switch statement, and the ? operator. By non-looping, I mean that the actions initiated by the statement take place and program flow then moves on, whereas looping conditionals execute code over and over until a condition has been met.

The if Statement

One way of thinking about program flow is to imagine it as a single-lane highway that you are driving along. It's pretty much a straight line, but now and then you encounter various signs telling you where to go.

In the case of an if statement, you could imagine coming across a detour sign that you have to follow if a certain condition is TRUE. If so, you drive off and follow the detour until you return to where it started and then continue on your way in your original direction. Or, if the condition isn't TRUE, you ignore the detour and carry on driving.



The contents of the if condition can be any valid PHP expression, including equality, comparison, tests for 0 and NULL, and even the values returned by functions (either builtin functions or ones that you write).

The actions to take when an if condition is TRUE are generally placed inside curly braces, { }. However, you can ignore the braces if you have only a single statement to execute. But if you always use curly braces, you'll avoid having to hunt down difficult-to-trace bugs, such as when you add an extra line to a condition and it doesn't get evaluated due to lack of braces.

In Example 1, imagine that it is the end of the month and all your bills have been paid, so you are performing some bank account maintenance.

Example 1. An if statement with curly braces

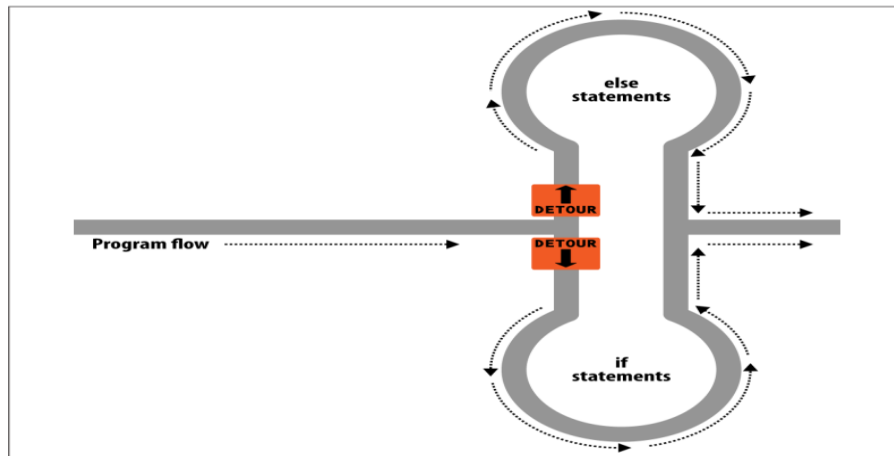
```
<?php
    if ($bank_balance < 100)
    {
        $money          = 1000;
        $bank_balance += $money;
    }
?>
```

In this example, you are checking your balance to see whether it is less than \$100 (or whatever your currency is). If so, you pay yourself \$1,000 and then add it to the balance. (If only making money were that simple!)

If the bank balance is \$100 or greater, the conditional statements are ignored and program flow skips to the next line (not shown).

The else Statement

Sometimes when a conditional is not TRUE, you may not want to continue on to the main program code immediately but might wish to do something else instead. This is where the else statement comes in. With it, you can set up a second detour on your highway, as in Figure .



With an if ... else statement, the first conditional statement is executed if the condition is TRUE. But if it's FALSE, the second one is executed. One of the two choices must be executed. Under no circumstance can both (or neither) be executed.

Example 2:

```
<?php
  if ($bank_balance < 100)
  {
    $money      = 1000;
    $bank_balance += $money;
  }
  else
  {
    $savings    += 50;
    $bank_balance -= 50;
  }
?>
```

In this example, now that you've ascertained that you have \$100 or more in the bank, the else statement is executed, by which you place some of this money into your savings account.

The elseif Statement

There are also times when you want a number of different possibilities to occur, based upon a sequence of conditions. You can achieve this using the elseif statement. As you might imagine, it is like an else statement, except that you

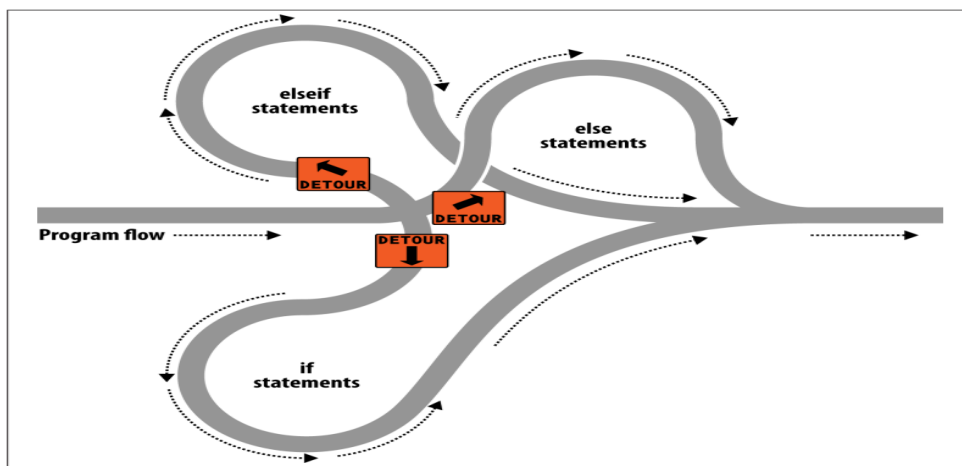
place a further conditional expression prior to the conditional code. In Example below , you can see a complete if ... elseif ... else construct.

Example3 :: An if ... elseif ... else statement with curly braces

```
<?php
if ($bank_balance < 100)
{
    $money        = 1000;
    $bank_balance += $money;
}
elseif ($bank_balance > 200)
{
    $savings      += 100;
    $bank_balance -= 100;
}
else
{
    $savings      += 50;
    $bank_balance -= 50;
}
?>
```

In the example, an elseif statement has been inserted between the if and else statements. It checks whether your bank balance exceeds \$200 and, if so, decides that you can afford to save \$100 of it this month.

Although I'm starting to stretch the metaphor a bit too far, you can imagine this as a multi-way set of detours (see below figure).



You may have as many elseif statements as you like. But as the number of elseif statements increases, you would probably be better advised to consider a switch statement if it fits your needs.

The switch Statement

The switch statement is useful in cases in which one variable or the result of an expression can have multiple values, which should each trigger a different function.

For example, consider a PHP-driven menu system that passes a single string to the main menu code according to what the user requests. Let's say the options are Home, About, News, Login, and Links, and we set the variable \$page to one of these, according to the user's input.

If we write the code for this using if ... elseif ... else, it might look like

Example 4 : A multiple-line if ... elseif ... statement

```
<?php
if      ($page == "Home")  echo "You selected Home";
elseif ($page == "About") echo "You selected About";
elseif ($page == "News")  echo "You selected News";
elseif ($page == "Login") echo "You selected Login";
elseif ($page == "Links") echo "You selected Links";
?>
```

If we use a switch statement, the code might look like :

Example 5:. A switch statement

```
<?php
switch ($page)
{
    case "Home":
        echo "You selected Home";
        break;
    case "About":
        echo "You selected About";
        break;
    case "News":
        echo "You selected News";
        break;
    case "Login":
        echo "You selected Login";
        break;
    case "Links":
        echo "You selected Links";
        break;
}
?>
```

As you can see, \$page is mentioned only once at the start of the switch statement. Thereafter, the case command checks for matches. When one occurs, the matching conditional statement is executed.

With switch statements, you do not use curly braces inside case commands. Instead, they commence with a colon and end with the break statement. The entire list of cases in the switch statement is enclosed in a set of curly braces, though.

Breaking out

If you wish to break out of the switch statement because a condition has been fulfilled, use the break command. This command tells PHP to break out of the switch and jump to the following statement.

If you were to leave out the break commands in above switch example and the case of Home evaluated to be TRUE, all five cases would then be executed. Or if \$page had the value News, then all the case commands from then on would execute. In fact, leaving out the break statement is a common error.

Default action

A typical requirement in switch statements is to fall back on a default action if none of the case conditions are met.

Example 6. A default statement to add to Example 5

```
default:
    echo "Unrecognized selection";
    break;
```

Although a break command is not required here because the default is the final substatement, and program flow will automatically continue to the closing curly brace, should you decide to place the default statement higher up it would definitely need a break command to prevent program flow from dropping into the following statements. Generally, the safest practice is to always include the break command.

Alternative syntax

If you prefer, you may replace the first curly brace in a switch statement with a single colon, and the final curly brace with an endswitch command, as in Example 7. However, this approach is not commonly used and is mentioned here only in case you encounter it in third-party code.

Example 7: Alternate switch statement syntax

```
<?php
switch ($page):
    case "Home":
        echo "You selected Home";
        break;

    // etc...

    case "Links":
        echo "You selected Links";
        break;
endswitch;
?>
```

The ? Operator:

One way of avoiding the verbosity of if and else statements is to use the more compact ternary operator, **?**, which is unusual in that it takes three operands rather than the typical two.

The **?** operator is passed an expression that it must evaluate, along with two statements to execute: one for when the expression evaluates to TRUE, the other for when it is FALSE.

Example 8 shows code we might use for writing a warning about the fuel level of a car to its digital dashboard.

Example 8. Using the ? operator

```
<?php
    echo $fuel <= 1 ? "Fill tank now" : "There's enough fuel";
?>
```

In this statement, if there is one gallon or less of fuel (i.e., if \$fuel is set to 1 or less), the string Fill tank now is returned to the preceding echo statement.

Otherwise, the string There's enough fuel is returned. You can also assign the value returned in a ? statement to a variable (see Example 8).

Example 8. Assigning a ? conditional result to a variable

```
<?php
    $enough = $fuel <= 1? FALSE: TRUE;
?>
```

Here \$enough will be assigned the value TRUE only when there is more than a gallon of fuel; otherwise, it is assigned the value FALSE.

It can be hard to read, because it often mixes multiple occurrences of the same variable. For instance, code such as the following is :

```
$saved = $saved >= $new ? $saved : $new;
```

If you take it apart carefully, you can figure out what this code does:

```
$saved =           // Set the value of $saved to...
    $saved >= $new // Check $saved against $new
    ?             // Yes, comparison is true ...
    $saved        // ... so assign the current value of $saved
    :             // No, comparison is false ...
    $new;         // ... so assign the value of $new
```

It's a concise way to keep track of the largest value that you've seen as a program progresses. You save the largest value in \$saved and compare it to \$new each time you get a new value. **Programmers familiar with the ? operator find it more convenient than if statements for such short comparisons. When not used for writing compact code, it is typically used to make some decision inline, such as when you are testing whether a variable is set before passing it to a function.**

Looping

Often you may want a program to repeat the same sequence of code again and again until something happens, such as a user inputting a value or reaching a natural end. PHP's various loop structures provide the perfect way to do this.

WHILE LOOPS

A loop that continuously checks the fuel level as you drive, using a while loop (Example 9).

Example 9. A while loop

```
<?php
    $fuel = 10;

    while ($fuel > 1)
    {
        // Keep driving ...
        echo "There's enough fuel";
    }
?>
```

For another example of a while loop that displays the 12 times table 4-29.

Example 10. A while loop to print the 12 times table

```
<?php
    $count = 1;

    while ($count <= 12)
    {
        echo "$count times 12 is " . $count * 12 . "<br>";
        ++$count;
    }
?>
```

Here the variable \$count is initialized to a value of 1, then a while loop is started with the comparative expression \$count <= 12. This loop will continue executing until the variable is greater than 12.

The output from this code is as follows:

1 times 12 is 12

2 times 12 is 24

3 times 12 is 36 and so on...

But there is a much neater way this loop can be written,

Example 11. A shortened version of Example10

```
<?php
    $count = 0;
    while (++$count <= 12)
    {
        echo "$count times 12 is " . $count * 12 . "<br>";
    }
?>
```

In this example, it was possible to remove the `++$count` statement from inside the while loop and place it directly into the conditional expression of the loop. What now happens is that PHP encounters the variable `$count` at the start of each iteration of the loop and, noticing that it is prefaced with the increment operator, **first increments the variable and only then compares it to the value 12**. You can therefore see that `$count` now has to be initialized to 0, not 1, because it is incremented as soon as the loop is entered. If you keep the initialization at 1, only results between 2 and 12 will be output.

DO ... WHILE LOOPS

A slight variation to the while loop is the **do ... while loop, used when you want a block of code to be executed at least once and made conditional only after that**. Example 12 shows a modified version of the code for the 12 times table that uses such a loop.

Example 12. A do ... while loop for printing the times table for 12

```
<?php
    $count = 1;
    do
        echo "$count times 12 is " . $count * 12 . "<br>";
    while (++$count <= 12);
?>
```

Notice how we are back to initializing \$count to 1 (rather than 0) because the code is being executed immediately, without an opportunity to increment the variable.

Of course, if you have more than a single statement inside a do ... while loop, remember to use curly braces, as in Example 13.

Example 13. Expanding Example 12 to use curly braces

```
<?php
    $count = 1;
    do {
        echo "$count times 12 is " . $count * 12;
        echo "<br>";
    } while (++$count <= 12);
?>
```

FOR LOOPS

The final kind of loop statement, **the for loop**, is also the most powerful, as it **combines the abilities to set up variables as you enter the loop, test for conditions while iterating loops, and modify variables after each iteration.**

Example 14 shows how you could write the multiplication table program with a for loop.

Example 14. Outputting the times table for 12 from a for loop

```
<?php
    for ($count = 1 ; $count <= 12 ; ++$count)
        echo "$count times 12 is " . $count * 12 . "<br>";
?>
```

Each for statement takes three parameters:

- An initialization expression
- A condition expression
- A modification expression

These are separated by semicolons like this: for (expr1 ; expr2 ; expr3). At the start of the first iteration of the loop, the initialization expression is executed. In

the case of the times table code, \$count is initialized to the value 1. Then, each time around the loop, the condition expression (in this case, \$count <= 12) is tested, and the loop is entered only if the condition is TRUE. Finally, at the end of each iteration, the modification expression is executed. In the case of the times table code, the variable \$count is incremented.

All this structure neatly removes any requirement to place the controls for a loop within its body, freeing it up just for the statements you want the loop to perform.

Example 15. The for loop from Example 14 with added curly braces

```
<?php
    for ($count = 1 ; $count <= 12 ; ++$count)
    {
        echo "$count times 12 is " . $count * 12;
        echo "<br>";
    }
?>
```

Let's compare when to use for and while loops. The for loop is explicitly designed around a single value that changes on a regular basis. Usually you have a value that increments, as when you are passed a list of user choices and want to process each choice in turn. But you can transform the variable any way you like.

A more complex form of the for statement even lets you perform multiple operations in each of the three parameters:

```
for ($i = 1, $j = 1 ; $i + $j < 10 ; $i++ , $j++)
{
    // ...
}
```

That's complicated and not recommended for first-time users. The key is to distinguish commas from semicolons. The three parameters must be separated by semicolons. Within each parameter, multiple statements can be separated by commas.

In the previous example, the first and third parameters each contain two statements:

```
$i = 1, $j = 1 // Initialize $i and $j
```

```
$i + $j < 10 // Terminating condition
```

```
$i++ , $j++ // Modify $i and $j at the end of each iteration
```

When is a while statement more appropriate than a for statement?

When your condition doesn't depend on a simple, regular change to a variable. For instance, if you want to check for some special input or error and end the loop when it occurs, use a while statement.

Breaking Out of a Loop

Break command can be used to break out of a for loop. This step can be necessary when, for example, one of your statements returns an error and the loop cannot continue executing safely.

Example 16. Writing a file using a for loop with error trapping

```
<?php
    $fp = fopen("text.txt", 'wb');
    for ($j = 0 ; $j < 100 ; ++$j)
    {
        $written = fwrite($fp, "data");
        if ($written == FALSE)
            break;
    }
    fclose($fp);
?>
```

The first line opens the file text.txt for writing in binary mode, and then returns a pointer to the file in the variable \$fp, which is used later to refer to the open file.

The loop then iterates 100 times (from 0 to 99) writing the string data to the file. After each write, the variable \$written is assigned a value by the fwrite function representing the number of characters correctly written. **But if there is an error, the fwrite function assigns the value FALSE. The behavior of fwrite makes it easy for the code to check the variable \$written to see whether it is set to FALSE and, if so, to break out of the loop to the following statement closing the file.**

The pair of inner loop statements can be shortened to the following single statement:

```
if (!fwrite($fp, "data")) break;
```

The code nested more than one layer deep that you need to break out of, you can follow the break command with a number to indicate how many levels to break out of, like this:

break 2;

The continue Statement

The continue statement is a little like a break statement, except that it instructs PHP to stop processing the current loop and to move right to its next iteration. So, instead of breaking out of the whole loop, PHP exits only the current iteration.

This approach can be useful in cases where you know there is no point continuing execution within the current loop and you want to save processor cycles or prevent an error from occurring by moving right along to the next iteration of the loop.

Example 17. Trapping division-by-zero errors using continue

```
<?php
    $j = 10;
    while ($j > -10)
    {
        $j--;
        if ($j == 0) continue;
        echo (10 / $j) . "<br>";
    }
?>
```

For all values of \$j between 10 and -10, with the exception of 0, the result of calculating 10 divided by \$j is displayed. But for the particular case of \$j being 0, the continue statement is issued and execution skips immediately to the next iteration of the loop.

Chapter 2 Arrays in PHP

What is an array?

An array can hold multiple, separate pieces of information. An array is therefore like a list of values, each value being a string or a number or even another array. Arrays are structured as a series of keyvalue pairs, where one pair is an item or element of that array. For each item in the list, there is a key (or index) associated with it.

PHP supports two kinds of arrays:

- Indexed, which use numbers as the keys (Table 2.3).
- Associative, which use strings as keys (Table 2.4).

TABLE 2.3 Array Example 1: \$artists

Key	Value
0	The Mynabirds
1	Jeremy Messersmith
2	The Shins
3	Iron and Wine
4	Alexi Murdoch

TABLE 2.4 Array Example 2: \$states

Key	Value
MD	Maryland
PA	Pennsylvania
IL	Illinois
MO	Missouri
IA	Iowa

As in most programming languages, **with indexed arrays, arrays will begin with the first index at 0, unless you specify the keys explicitly.**

An array follows the same naming rules as any other variable. This means that, offhand, you might not be able to tell that \$var is an array as opposed to a string or number. The important syntactical difference arises when accessing individual array elements.

- **To refer to a specific value in an array, start with the array variable name, followed by the key within square brackets:**

```
$band = $artists[0]; // The Mynabirds  
echo $states['MD']; // Maryland
```

The array keys are used like other values in PHP: **numbers (e.g., 0) are never quoted, whereas strings (MD) must be.**

- Arrays use a different syntax than other variables, and can contain multiple values, printing them can be trickier.

```
echo "My list of states: $states";
```

output:

My list of states: Array

Attempting to print an array using only the variable's name results in the word Array being printed.

- **Printing an individual element's value is simple if it uses indexed (numeric) keys:**

```
echo "The first artist is $artists[0].";
```

Output: The first artist is The Mynabirds.

- **if the array uses strings for the keys, the quotes used to surround the key will muddle the syntax. The following code will cause a parse error :**

```
echo "IL is $states['IL']."; // BAD!
```

Output:

Parse error: syntax error, unexpected T_ENCAPSED_AND_WHITESPACE, expecting T_STRING or T_VARIABLE or T_NUM_STRING in /Users/larryullman/Sites/phpmysql4/test.php on line 13

Attempting to print an element in an associative array without using curly braces results in a parse error

- **To fix this, wrap the array name and key in curly braces when an array uses strings for its keys :**

```
echo "IL is {$states['IL']}.";
```

output:

IL is Illinois.

Superglobal Arrays

PHP includes several predefined arrays called the superglobal variables. They are: `$_GET`, `$_POST`, `$_REQUEST`, `$_SERVER`, `$_ENV`, `$_SESSION`, and `$_COOKIE`.

- The `$_GET` variable is where PHP stores all of the values sent to a PHP script via the GET method (possibly but not necessarily from an HTML form).
- `$_POST` stores all of the data sent to a PHP script from an HTML form that uses the POST method.
- `$_GET` , `$_POST` , `$_COOKIE`—are subsets of `$_REQUEST`.
- `$_SERVER`, stores information about the server PHP is running on, as does `$_ENV`.

One aspect of good security and programming is to be precise when referring to a variable. This means that, although you can use `$_REQUEST` to access form data submitted through the POST method, `$_POST` would be more accurate.

To use arrays:

```
1  <!DOCTYPE html PUBLIC "-//W3C//DTD XHTML 1.0 Transitional//EN" "http://www.w3.org/TR/xhtml1/
   DTD/xhtml1-transitional.dtd">
2  <html xmlns="http://www.w3.org/1999/xhtml" xml:lang="en" lang="en">
3  <head>
4      <meta http-equiv="Content-Type" content="text/html; charset=utf-8" />
5      <title>Form Feedback</title>
6  </head>
7  <body>
8  <?php # Script 2.5 - handle_form.php #4
9
10 // Print the submitted information:
11 if ( !empty($_POST['name']) && !empty($_POST['comments']) && !empty($_POST['email']) ) {
12     echo "<p>Thank you, <b>{$_POST['name']}</b>, for the following comments:<br />
13     <tt>{$_POST['comments']}</tt></p>
14     <p>We will reply to you at <i>{$_POST['email']}</i>.</p>\n";
15 } else { // Missing form value.
16     echo '<p>Please go back and fill out the form again.</p>';
17 }
18 ?>
19 </body>
20 </html>
```

Creating arrays:

There are two primary ways to define an array.

First method: You could add an element at a time to build one:

Example: Numerically indexed array.

```
$band[] = 'Jemaine';  
$band[] = 'Bret';  
$band[] = 'Murray';
```

As arrays are indexed starting at 0, \$band[0] has a value of Jemaine; \$band[1], Bret, and \$band[2], Murray

Alternatively, you can specify the key when adding an element. But it's important to understand that if you specify a key and a value already exists indexed with that same key, the new value will overwrite the existing one:

Example: Associative Arrays

```
$band['fan'] = 'Mel';  
$band['fan'] = 'Dave'; // New value  
$fruit[2] = 'apple';  
$fruit[2] = 'orange'; // New value
```

Second Method:

Instead of adding one element at a time, you can use the array() function to build an entire array in one step:

Example:

```
$states = array (   'IA' => 'Iowa',  
                   'MD' => 'Maryland'  
                 );
```

The array() function can be used whether or not you explicitly set the key:

Example:

```
$artists = array ('Clem Snide', 'Shins', 'Eels');
```

if you set the first numeric key value, the added values will be keyed incrementally thereafter:

Example:

```
$days = array (1 => 'Sun', 'Mon', 'Tue');  
echo $days[3];           // Tue
```

The array() function is also used to initialize an array, prior to referencing it:

Example: \$tv = array();
 \$tv[] = 'Flight of the Conchords';

Initializing an array (or any variable) in PHP isn't required, but it makes for clearer code and can help avoid errors.

if you want to create an array of sequential numbers, you can use the range() function: \$ten = range (1, 10);

Accessing entire arrays:

To access every array element, use the foreach loop:

```
foreach ($array as $value)
{
    // Do something with $value. }
```

The foreach loop will iterate through every element in \$array, assigning each element's value to the \$value variable.

To access both the keys and values, use

```
foreach ($array as $key => $value)
{
    echo "The value at $key is $value.";
}
```

To create and access arrays:

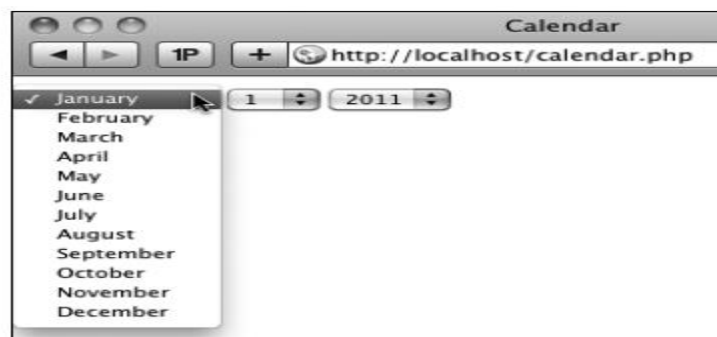
Example:

```

1  <!DOCTYPE html PUBLIC "-//W3C//DTD
   XHTML 1.0 Transitional//EN"
   "http://www.w3.org/TR/xhtml1/DTD/
   xhtml1-transitional.dtd">
2  <html xmlns="http://www.w3.org/1999/
   xhtml" xml:lang="en" lang="en">
3  <head>
4      <meta http-equiv="Content-Type"
       content="text/html; charset=utf-8" />
5      <title>Calendar</title>
6  </head>
7  <body>
8      <form action="calendar.php"
       method="post">
9      <?php # Script 2.6 - calendar.php
10
11     // This script makes three pull-down
       menus
12     // for an HTML form: months, days,
       years.
13
14     // Make the months array:
15     $months = array (1 => 'January',
       'February', 'March', 'April', 'May',
       'June', 'July', 'August', 'September',
       'October', 'November', 'December');
16
17     // Make the days and years arrays:
18     $days = range (1, 31);
19     $years = range (2011, 2021);
20
21     // Make the months pull-down menu:
22     echo '<select name="month">';
23     foreach ($months as $key =>
       $value) {
24         echo "<option value=\"\$key\">
       $value</option>\n";
25     }
26     echo '</select>';
27
28     // Make the days pull-down menu:
29     echo '<select name="day">';
30     foreach ($days as $value) {
31         echo "<option value=\"\$value\">
       $value</option>\n";
32     }
33     echo '</select>';
34
35     // Make the years pull-down menu:
36     echo '<select name="year">';
37     foreach ($years as $value) {
38         echo "<option value=\"\$value\">
       $value</option>\n";
39     }
40     echo '</select>';
41
42     ?>
43 </form>
44 </body>
45 </html>

```

Output:



F These pull-down menus will be created using arrays and the **foreach** loop.

TIP To determine the number of elements in an array, use `count( )`:

```
$num = count($array);
```

TIP The `range( )` function can also create an array of sequential letters:

```
$alphabet = range ('a', 'z');
```

TIP An array's key can be multiple-worded strings, such as *first name* or *phone number*.

TIP The `is_array( )` function confirms that a variable is of the array type.

TIP If you see an *Invalid argument supplied for foreach()* error message, that means you are trying to use a `foreach` loop on a variable that is not an array.

Multidimensional arrays:

An array consisting of other arrays —creates a multidimensional array.

Example:

```
$primes = array(2, 3, 5, 7, ...);  
$sphenic = array(30, 42, 66, 70, ...);
```

These two arrays could be combined into one multidimensional array :

```
$numbers = array ('Primes' => $primes,  
                 'Sphenic' => $sphenic  
                );
```

`$numbers` is a multidimensional array.

To access the prime numbers sub-array : `$numbers['Primes']`.

To access the prime number 5:

```
$numbers['Primes'][2] (it's the third element in the array, but the array  
starts indexing at 0).
```

To print out one of these values, surround the whole construct in curly braces:

```
echo "The first sphenic number is { $numbers['Sphenic'][0] }.";
```

Can access multidimensional arrays using the foreach loop, nesting one inside another if necessary:

```
foreach ($numbers as $k=>$v )  
{  
    echo $k;  
    echo "<br>";  
    foreach($v as $m)  
    {  
        echo "Element is $m";  
        echo "<br>";  
    }  
}
```

Output:

```
primes  
Element is 2  
Element is 3  
Element is 5  
Element is 7  
sphenic  
Element is 30  
Element is 42  
Element is 66  
Element is 70
```

Example 6-10. Creating a multidimensional associative array

```

<?php
    $products = array(

        'paper' => array(

            'copier' => "Copier & Multipurpose",
            'inkjet' => "Inkjet Printer",
            'laser'  => "Laser Printer",
            'photo'  => "Photographic Paper"),

        'pens' => array(

            'ball'   => "Ball Point",
            'hilite' => "Highlighters",
            'marker' => "Markers"),

        'misc' => array(

            'tape'   => "Sticky Tape",
            'glue'   => "Adhesives",
            'clips'  => "Paperclips"
        )
    );

    echo "<pre>";

    foreach($products as $section => $items)
        foreach($items as $key => $value)
            echo "$section:\t$key\t($value)<br>";

echo "</pre>";

?>

```

The echo statement makes use of the PHP escape character `\t`, which outputs a tab. Although tabs are not normally significant to the web browser, I let them be used for layout by using the `<pre> ... </pre>` tags, which tell the web browser to format the text as preformatted and monospaced, and *not* to ignore whitespace characters such as tabs and line feeds. The output from this code looks like the following:

```

paper:  copier  (Copier & Multipurpose)
paper:  inkjet  (Inkjet Printer)
paper:  laser   (Laser Printer)
paper:  photo   (Photographic Paper)
pens:    ball   (Ball Point)
pens:    hilite (Highlighters)
pens:    marker (Markers)
misc:    tape   (Sticky Tape)
misc:    glue   (Adhesives)
misc:    clips  (Paperclips)

```

You can directly access a particular element of the array using square brackets, like this:

```
echo $products['misc']['glue'];
```

which outputs the value Adhesives.

Example 6-11. Creating a multidimensional numeric array

```

<?php
    $chessboard = array(
        array('r', 'n', 'b', 'q', 'k', 'b', 'n', 'r'),
        array('p', 'p', 'p', 'p', 'p', 'p', 'p', 'p'),
        array(' ', ' ', ' ', ' ', ' ', ' ', ' ', ' '),
        array(' ', ' ', ' ', ' ', ' ', ' ', ' ', ' '),
        array(' ', ' ', ' ', ' ', ' ', ' ', ' ', ' '),
        array(' ', ' ', ' ', ' ', ' ', ' ', ' ', ' '),
        array('P', 'P', 'P', 'P', 'P', 'P', 'P', 'P'),
        array('R', 'N', 'B', 'Q', 'K', 'B', 'N', 'R')
    );

    echo "<pre>";

    foreach($chessboard as $row)
    {
        foreach ($row as $piece)
            echo "$piece ";

        echo "<br>";
    }

    echo "</pre>";
?>

```

Output:

```

r n b q k b n r
p p p p p p p p

```

```

P P P P P P P P
R N B Q K B N R

```


Chapter 3 Types of array

2 types of array are: Numerically Indexed Arrays and Associative array.

(In chapter 2 we have discussed briefly about these two types of array)

Numerically Indexed Arrays

Example 3.1: Adding items to an array

```
<?php
```

```
    $paper[] = "Copier";
```

```
    $paper[] = "Inkjet";
```

```
    $paper[] = "Laser";
```

```
    $paper[] = "Photo";
```

```
    print_r($paper);
```

```
?>
```

In this example, each time you assign a value to the array \$paper, the first empty location within that array is used to store the value, and a pointer internal to PHP is incremented to point to the next free location, ready for future insertions. The **print_r function (which prints out the contents of a variable, array, or object) is used to verify that the array has been correctly populated.** It prints out the following:

Array

(

[0] => Copier

[1] => Inkjet

[2] => Laser

[3] => Photo

)

The previous code could also have been written as shown in Example 3-2, where the exact location of each item within the array is specified.

Example 3-2. Adding items to an array using explicit locations

```
<?php
```

```
$paper[0] = "Copier";
```

```
$paper[1] = "Inkjet";
```

```
$paper[2] = "Laser";  
$paper[3] = "Photo";  
print_r($paper);  
?>
```

Example 3-3 shows how you might print out the various types of paper the website offers using a for loop

Example 3-3. Adding items to an array and retrieving them

```
<?php  
    $paper[] = "Copier";  
    $paper[] = "Inkjet";  
    $paper[] = "Laser";  
    $paper[] = "Photo";  
    for ($j = 0 ; $j < 4 ; ++$j)  
        echo "$j: $paper[$j]<br>";  
?>
```

This example prints out the following:

```
0: Copier  
1: Inkjet  
2: Laser  
3: Photo
```

Associative Arrays

Keeping track of array elements by index works just fine, but can require extra work in terms of remembering which number refers to which product. It can also make code hard for other programmers to follow.

This is where associative arrays come into their own. Using them, you can reference the items in an array by name rather than by number.

Example 3-4. Adding items to an associative array and retrieving them

```
<?php  
    $paper['copier'] = "Copier & Multipurpose";  
    $paper['inkjet'] = "Inkjet Printer";  
    $paper['laser'] = "Laser Printer";  
    $paper['photo'] = "Photographic Paper";
```

```
    echo $paper['laser'];  
?>
```

In place of a number, each item now has a unique name that you can use to reference it elsewhere, as with the echo statement—which simply prints out Laser Printer. **The names (copier, inkjet, etc.) are called indexes or keys and the items assigned to them (such as “Laser Printer”) are called values.**

This very powerful feature of PHP is often used when you are extracting information from XML and HTML. For example, an HTML parser such as those used by a search engine could place all the elements of a web page into an associative array whose names reflect the page’s structure:

```
$html['title'] = "My web page";  
  
$html['body'] = "... body of web page ...";
```

Assignment Using the array Keyword

Example 3-5. Adding items to an array using the array keyword

```
<?php  
    $p1 = array("Copier", "Inkjet", "Laser", "Photo");  
    echo "p1 element: " . $p1[2] . "<br>";  
    $p2 = array('copier' =>"Copier & Multipurpose",  
                'inkjet' =>"Inkjet Printer",  
                'laser' =>"Laser Printer",  
                'photo' =>"Photographic Paper");  
    echo "p2 element: " . $p2['inkjet'] . "<br>";  
?>  
  
output: p1 element: Laser  
p2 element: Inkjet Printer
```

You can verify that \$p1 and \$p2 are different types of array, because both of the following commands, when appended to the code, will cause an Undefined index or Undefined offset error, as the array identifier for each is incorrect:

```
echo $p1['inkjet']; // Undefined index
```

```
echo $p2[3]; // Undefined offset
```

The foreach ... as Loop

The creators of PHP have gone to great lengths to make the language easy to use. So, not content with the loop structures already provided, they added another one especially for arrays: **the foreach ... as loop. Using it, you can step through all the items in an array, one at a time, and do something with them.**

The process starts with the first item and ends with the last one, so you don't even have to know how many items there are in an array.

Example 3-6 shows how foreach ... as can be used to rewrite Example 3-3.

Example 3-6. Walking through a numeric array using foreach ... as

```
<?php
    $paper = array("Copier", "Inkjet", "Laser", "Photo");
    $j = 0;
    foreach($paper as $item)
    {
        echo "$j: $item<br>";
        ++$j;
    }
?>
```

When PHP encounters a foreach statement, it takes the first item of the array and places it in the variable following the as keyword; and each time control flow returns to the foreach, the next array element is placed in the as keyword. In this case, the variable \$item is set to each of the four values in turn in the array \$paper. Once all values have been used, execution of the loop ends. The output from this code is exactly the same as Example 3-3.

Example 3-7. Walking through an associative array using foreach ... as

```
<?php

    $paper = array('copier' => "Copier & Multipurpose",
                  'inkjet' => "Inkjet Printer",
                  'laser' => "Laser Printer",
```

```
'photo' =>"Photographic Paper");  
foreach($paper as $item => $description)  
  
    echo "$item: $description<br>";  
  
?>
```

Remember that associative arrays do not require numeric indexes, so the variable `$j` is not used in this example. Instead, each item of the array `$paper` is fed into the key/value pair of variables `$item` and `$description`, from which they are printed out.

The displayed result of this code is as follows:

```
copier: Copier & Multipurpose  
inkjet: Inkjet Printer  
laser: Laser Printer  
photo: Photographic Paper
```

list and each:

As an alternative syntax to `foreach ... as`, you can use the `list` function in conjunction with the `each` function, as in Example 3-8.

Example 3-8. Walking through an associative array using `each` and `list`

```
<?php  
  
$paper = array('copier' =>"Copier & Multipurpose",  
'inkjet' =>"Inkjet Printer",  
'laser' =>"Laser Printer",  
'photo' =>"Photographic Paper");  
while (list($item, $description) = each($paper))  
    echo "$item: $description<br>";  
  
?>
```

In this example, a while loop is set up and will continue looping until the `each` function returns a value of `FALSE`. The `each` function acts like `foreach`: it returns an array containing a key/value pair from the array `$paper` and then moves its

built-in pointer to the next pair in that array. When there are no more pairs to return, each returns FALSE.

The list function takes an array as its argument (in this case, the key/value pair returned by the function each) and then assigns the values of the array to the variables listed within parentheses.

You can see how list works a little more clearly in Example 3-9, where an array is created out of the two strings Alice and Bob and then passed to the list function, which assigns those strings as values to the variables \$a and \$b.

Example 3-9. Using the list function

```
<?php  
  
list($a, $b) = array('Alice', 'Bob');  
  
echo "a=$a b=$b";  
  
?>
```

The output from this code is:

a=Alice b=Bob

Using Array Functions:

is_array:

Arrays and variables share the same namespace. This means that you cannot have a string variable called \$fred and an array also called \$fred. If you're in doubt and your code needs to check whether a variable is an array, you can use the is_array function like this:

```
echo (is_array($fred)) ? "Is an array" : "Is not an array";
```

Note that if \$fred has not yet been assigned a value, an Undefined variable message will be generated.

Count:

Sometimes you need to know exactly how many elements there are in your array, particularly if you will be referencing them directly. **To count all the elements in the top level of an array, use a command such as the following:**

```
echo count($fred);
```

To know how many elements there are altogether in a multidimensional array:

```
echo count($fred, 1);
```

The second parameter is optional and sets the mode to use. It should be either a 0 to limit counting to only the top level, or 1 to force recursive counting of all subarray elements too.

Sort:

Sorting is so common that PHP provides a built-in function. In its simplest form, you would use it like this:

```
sort($fred);  
echo $chessboard[7][3];
```

Sort will act directly on the supplied array rather than returning a new array of sorted elements. Instead, it returns TRUE on success and FALSE on error and also supports a few flags.

The main two sorting to be made either numerically or as strings, like this:

```
sort($fred, SORT_NUMERIC);  
sort($fred, SORT_STRING);
```

You can also sort an array in reverse order using the rsort function, like this:

```
rsort($fred, SORT_NUMERIC);  
rsort($fred, SORT_STRING);
```

Shuffle:

Using Shuffle function the elements of an array to be put in random order, such as when you're creating a game of playing cards:

```
shuffle($cards);
```

Like sort, shuffle acts directly on the supplied array and returns TRUE on success or FALSE on error.

explode

This is a very useful function with which you can take a string containing several items separated by a single character (or string of characters) and then place each of these items into an array. One handy example is to split up a sentence into an array containing all its words, as in Example 3-12.

Example 3-12. Exploding a string into an array using spaces

```
<?php
    $temp = explode(" ", "This is a sentence with seven words");
    print_r($temp);    ?>
```

This example prints out the following (on a single line when viewed in a browser):

Array

```
(
  [0] => This
  [1] => is
  [2] => a
  [3] => sentence
  [4] => with
  [5] => seven
  [6] => words
)
```

The first parameter, the delimiter, need not be a space or even a single character.

Example 3-13. Exploding a string delimited with *** into an array

```
<?php
    $temp = explode('***', "A***sentence***with***asterisks");
    print_r($temp);
?>
```


The code in Example 3-13 prints out the following:

Array

```
(  
  [0] => A  
  [1] => sentence  
  [2] => with  
  [3] => asterisks  
)
```

Extract:

Sometimes it can be convenient to turn the key/value pairs from an array into PHP variables. One such time might be when you are processing the `$_GET` or `$_POST` variables as sent to a PHP script by a form.

When a form is submitted over the Web, the web server unpacks the variables into a global array for the PHP script. If the variables were sent using the GET method, they will be placed in an associative array called `$_GET`; if they were sent using POST, they will be placed in an associative array called `$_POST`.

You could, of course, walk through such associative arrays in the manner shown in the examples so far. However, sometimes you just want to store the values sent into variables for later use. In this case, you can have PHP do the job automatically for you:

```
extract($_GET);
```

So, for example, if the query string parameter `q` is sent to a PHP script along with the associated value `Hi` there, a new variable called `$q` will be created and assigned that value.

Be careful with this approach, though, because if any extracted variables conflict with ones that you have already defined, your existing values will be overwritten. **To avoid this possibility, you can use one of the many additional parameters available to this function, like this:**

```
extract($_GET, EXTR_PREFIX_ALL, 'fromget');
```

In this case, all the new variables will begin with the given prefix string followed by an underscore, so \$q will become \$fromget_q. I strongly recommend that you use this version of the function when handling the \$_GET and \$_POST arrays, or any other array whose keys could be controlled by the user, because malicious users could submit keys chosen deliberately to overwrite commonly used variable names and compromise your website.

Compact:

Compact, the inverse of extract, to create an array from variables and their values.

Example 3-14. Using the compact function

```
<?php
    $fname = "Doctor";
    $sname = "Who";
    $planet = "Gallifrey";
    $system = "Gridlock";
    $constellation = "Kasterborous";
    $contact = compact('fname', 'sname', 'planet', 'system', 'constellation');
    print_r($contact);
?>
```

The result of running Example 3-14 is:

```
Array
(
    [fname] => Doctor
    [sname] => Who
    [planet] => Gallifrey
    [system] => Gridlock
    [constellation] => Kasterborous
)
```

Note: compact requires the variable names to be supplied in quotes, not preceded by a \$ symbol. This is because compact is looking for a list of variable names.

Another use of this function is for debugging, when you wish to quickly view several variables and their values, as in Example 3-15.

Example 3-15. Using compact to help with debugging

```
<?php
    $j = 23;
    $temp = "Hello";
    $address = "1 Old Street";
    $age = 61;
    print_r(compact(explode(' ', 'j temp address age')));
?>
```

This works by using the explode function to extract all the words from the string into an array, which is then passed to the compact function, which in turn returns an array to print_r, which finally shows its contents.

If you copy and paste the print_r line of code, you only need to alter the variables named there for a quick printout of a group of variables' values. In this example, the output is:

```
Array
(
    [j] => 23
    [temp] => Hello
    [address] => 1 Old Street
    [age] => 61
)
```

reset:

When the foreach ... as construct or the each function walks through an array, it keeps an internal PHP pointer that makes a note of which element of the array it should return next. If your code ever needs to return to the start of an array, you can issue reset, which also returns the value of that element.

Examples of how to use this function are:

```
reset($fred);           // Throw away return value
```

```
$item = reset($fred); // Keep first element of the array in $item end
```

end:

As with reset, you can move PHP's internal array pointer to the final element in an array using the end function, which also returns the value of the element, and can be used as in these examples:

```
end($fred);
```

```
$item = end($fred);
```

Including and Requiring Files

You are likely to start building a library of functions that you think you will need again. You'll also probably start using libraries created by other programmers. There's no need to copy and paste these functions into your code. You can save them in separate files and use include or require to pull them in.

There are two types of commands to perform this action: include and require.

The include (or require) statement takes all the text/code/markup that exists in the specified file and copies it into the file that uses the include statement.

The include Statement :

Using include, you can tell PHP to fetch a particular file and load all its contents. It's as if you pasted the included file into the current file at the insertion point.

Example : Including a PHP file

```
<?php
    include "library.php";
    // your code
?>
```

Using include_once :

Each time you issue the include directive, it includes the requested file again, even if you've already inserted it. For instance, suppose that library.php contains a lot of useful functions, so you include it in your file, but also include

another library that includes library.php. Through nesting, you've inadvertently included library.php twice. This will produce error messages, because you're trying to define the same constant or function multiple times. To avoid issue should use include_once instead.

```
<?php
    include_once "library.php";
    // your code
?>
```

Then, whenever another include or include_once is encountered, if it has already been executed, it will be completely ignored. To determine whether the file has already been executed, the absolute file path is matched after all relative paths are resolved and the file is found in your include path.

Using require and require_once:

A potential problem with include and include_once is that PHP will only attempt to include the requested file. Program execution continues even if the file is not found. When it is absolutely essential to include a file, require it.

```
<?php
    require_once "library.php";
    // your code
?>
```

Implicit and Explicit Casting

PHP is a loosely typed language that allows you to declare a variable and its type simply by using it. It also automatically converts values from one type to another whenever required. This is called implicit casting.

Example 4-37. This expression returns a floating-point number

```
<?php
    $a = 56;
    $b = 12;
    $c = $a / $b;

    echo $c;
?>
```

But what if we had wanted `$c` to be an integer instead? There are various ways in which we could achieve this, one of which is to force the result of `$a/$b` to be cast to an integer value using the integer cast type `(int)`, like this:

```
$c = (int) ($a / $b);
```

This is called *explicit casting*. Note that in order to ensure that the value of the entire expression is cast to an integer, we place the expression within parentheses. Otherwise, only the variable `$a` would have been cast to an integer—a pointless exercise, as the division by `$b` would still have returned a floating-point number.

Table 4-6. PHP's cast types

Cast type	Description
<code>(int)</code> (integer)	Cast to an integer by dropping the decimal portion.
<code>(bool)</code> (boolean)	Cast to a Boolean.
<code>(float)</code> (double) (real)	Cast to a floating-point number.
<code>(string)</code>	Cast to a string.
<code>(array)</code>	Cast to an array.
<code>(object)</code>	Cast to an object.